

Laboratory work No. 2
«Linear Algebraic Equations 12.39»

Submitted by
Farrakhetdinova Azaliya
05/12/2025

Problem statement

The steady-state distribution of temperature on a heated plate can be modeled by the *Laplace equation*,

$$0 = \frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2}$$

If the plate is represented by a series of nodes (Fig. 1), centered finite-divided differences can be substituted for the second derivatives, which results in a system of linear algebraic equations. Use the Gauss-Seidel method to solve for the temperatures of the nodes in Fig. 1.

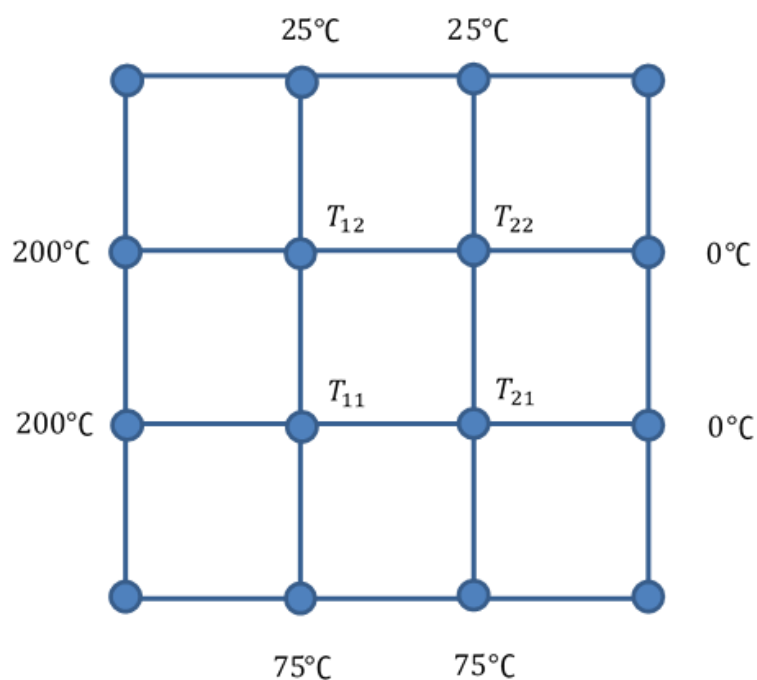


Fig. 1 – Heated plate with nodes

Mathematical model

Mathematical model of the problem consists of the Laplace equation

$$0 = \frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2}$$

The second derivatives can be approximated with centered finite-divided differences

$$\frac{\partial^2 T}{\partial x^2} \approx \frac{T(x+h) - 2T(x) + T(x-h)}{h^2}$$

After that we can obtain a system of linear algebraic equations.

Numerical method

Let system of linear equations be given in matrix form like represented below

$$A\vec{x} = \vec{b}$$

where A – is a matrix with dimension $[n \times n]$,

$\vec{x}, \vec{b}$ – are vectors with dimension $[n]$

Given system can be solved with iterative method

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left[b_i - \sum_{j=i}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right]$$

The convergence criterion of Gauss-Seidel method:

$$|a_{ii}| \geq \sum_{j \neq i}^n |a_{ij}|$$

Result of numerical solution

The main window of interface is shown on Fig.1. There are the input fields and nodes with solutions.

Gauss-Seidel Method

Choose value of step: $x = 1$, $y = 1$, Error = 0.0005 **Solve**

Choose value of temperature:

	$T = 25$	$T = 25$	
$T = 200$	$T = 93.74995525$	$T = 43.74997764$	$T = 0$
$T = 200$	$T = 106.2499776$	$T = 56.24998884$	$T = 0$
	$T = 75$	$T = 75$	

Fig.1 – Main window

Conclusion

For given steps of x and y , the program outputs the value corresponding to the equation's solution. The solution can be described from a physical point a view. The high temperature influences on neighbor nodes with less temperature and vice versa.

Appendix.

A. Main Window Code

```
import numpy as np
from PyQt5 import QtWidgets, uic

from solution import Solution

class MainWindow(QtWidgets.QMainWindow):

    def __init__(self):

        super(MainWindow, self).__init__()

        uic.loadUi('interface.ui', self)

        self.solve_button.clicked.connect(self.show_solution)

    def show_solution(self):

        self.x = float(self.x_edit.text())
        self.y = float(self.y_edit.text())
        self.Error = float(self.Error_edit.text())

        T_arr: np.ndarray = np.array([
            [0, float(self.T12_edit.text()),
             float(self.T13_edit.text()), 0],
            [float(self.T21_edit.text()), 0,
             float(self.T24_edit.text())],
            [float(self.T31_edit.text()), 0,
             float(self.T34_edit.text())],
            [0, float(self.T42_edit.text()),
             float(self.T43_edit.text()), 0]
        ])

        self.solution = Solution(T=T_arr,
                                x=self.x,
                                y=self.y,
                                Error=self.Error)

        T_solution = self.solution.solve()

        print('solved')

        self.T22_edit.setText(str(T_solution[0]))
        print('read')
        self.T23_edit.setText(str(T_solution[1]))
        self.T32_edit.setText(str(T_solution[2]))
        self.T33_edit.setText(str(T_solution[3]))

if __name__ == '__main__':
    app = QtWidgets.QApplication([])
    main = MainWindow()
    main.show()
    app.exec_()
```

B. Gauss-Seidel Method Solver Class

```
import numpy as np

from gauss_seidel_method import gauss_seidel

class Solution():

    """ Class for solution problem 12-39 with Gauss-Seidel Method """

    def __init__( self,
                    T: np.ndarray,
                    x: float,
                    y: float,
                    Error: float ):

        self.T_arr: np.ndarray = T # it's a matrix [4 x 4]
        self.x: float = x
        self.y: float = y
        self.Error: float = Error

        self.T_start: np.ndarray = np.zeros(shape=4, dtype=float) # it is a
        vector of unresolved T

        a = 1/pow(self.x, 2)
        b = 1/pow(self.y, 2)
        c = -2 * (a + b)

        self.A: np.ndarray = np.array([
            [c, a, b, 0],
            [a, c, 0, b],
            [b, 0, c, a],
            [0, b, a, c]
        ])

        self.b: np.ndarray = np.array([
            -(self.T_arr[1][0] * a + self.T_arr[0][1] * b),
            -(self.T_arr[1][3] * a + self.T_arr[0][2] * b),
            -(self.T_arr[2][0] * a + self.T_arr[3][1] * b),
            -(self.T_arr[2][3] * a + self.T_arr[3][2] * b),
        ])

    def solve(self):

        T_old: np.array
        T_new: np.array = np.copy(self.T_start)
        cur_error: float = 1e5

        while cur_error > self.Error:
            T_old = np.copy(T_new)
            T_new = gauss_seidel(T_old=T_old, A=self.A, b=self.b)
            cur_error = abs(T_old[0] - T_new[0])
            print("cur_error = ", cur_error)
            print("T_new = ", T_new)
            print("T_old = ", T_old)

        return T_new
```

C. Gauss-Seidel Method Implementation

```
import numpy as np

def gauss_seidel(
    T_old: np.ndarray,
    A: np.ndarray,
    b: np.ndarray ):

    """ returns new set (T_arr) """

    N: int = len(T_old)
    T_new: np.ndarray = np.zeros(shape=N, dtype=float)

    for i in range(N):
        T_new[i] = 1/A[i][i] * ( b[i] - np.sum(A[i][0:i] * T_new[0:i]) -
np.sum(A[i][i+1:N] * T_old[i+1:N]) )

    return T_new
```